

# AWS IAM (Identity and Access Management) – Complete Notes (Part 1)

**Author:** Dinesh Prajapat

**Topic:** AWS IAM Fundamentals to IAM Roles & Trust Policies

---

## Table of Contents

1. Introduction to IAM
  2. Authentication vs Authorization
  3. IAM Building Blocks
  4. IAM Users
  5. IAM Groups
  6. IAM Credentials
  7. Access Keys
  8. Multi-Factor Authentication (MFA)
  9. Password Policies
  10. Credential Rotation
  11. IAM Roles
  12. IAM User vs IAM Role
  13. Temporary Credentials
  14. Trust Policies
  15. Permission Policies
  16. Principal
  17. Real-World Architecture
  18. Security Best Practices
  19. Common Interview Questions
  20. Key Takeaways
- 

## 1. Introduction to IAM

### What is IAM?

AWS Identity and Access Management (IAM) is the AWS service used to securely manage access to AWS resources.

IAM answers two fundamental questions:

- **Who are you?**
- **What are you allowed to do?**

IAM acts as the security layer between users/services and AWS resources.

---

## Real-World Analogy

Imagine a company office.

- Employees have ID cards.
- Security guards verify identities.
- Different employees can access different rooms.

Similarly, IAM controls who can access AWS resources and what actions they can perform.

---

## Why IAM is Important

Without IAM:

- Anyone could access AWS resources.
- Data could be stolen.
- Resources could be deleted accidentally.
- Organizations would lose accountability.

With IAM:

- Secure authentication
  - Fine-grained permissions
  - Least privilege access
  - Auditing through CloudTrail
  - Better compliance and governance
- 

## 2. Authentication vs Authorization

### Authentication

Authentication means verifying identity.

Example:

Username + Password

or

Access Key + Secret Key

Question answered:

**Who are you?**

---

## Authorization

Authorization determines what an authenticated identity is allowed to do.

Example:

- Read S3 bucket
- Launch EC2 instance
- Create Lambda function

Question answered:

**What can you do?**

---

## AWS Authorization Flow

User / Service

↓

Authentication

↓

Authorization (IAM Policy Evaluation)

↓

AWS Resource

AWS always verifies identity first, then checks permissions.

---

## 3. IAM Building Blocks

IAM consists of four primary components:

## **IAM User**

Represents a permanent identity.

Examples:

- Developer
  - Administrator
  - Data Engineer
- 

## **IAM Group**

A collection of IAM Users.

Purpose:

Simplify permission management.

---

## **IAM Role**

A temporary identity assumed by trusted entities.

Used by:

- EC2
  - Lambda
  - ECS
  - AWS Glue
  - Cross-account access
  - IAM users (when assuming a role)
- 

## **IAM Policy**

A JSON document defining permissions.

Policies answer:

What actions are allowed or denied?

---

## 4. IAM Users

### Definition

An IAM User represents a permanent AWS identity.

Users can have:

- Username
  - Password
  - Access Keys
  - Policies
  - Group Membership
  - MFA Device
- 

### Important Facts

IAM Users have **no permissions by default**.

Permissions must be granted through:

- Managed Policies
  - Inline Policies
  - Group Membership
- 

### Console Access

Uses:

- Username
  - Password
  - MFA (recommended)
- 

### Programmatic Access

Uses:

- Access Key ID
- Secret Access Key

Used with:

- AWS CLI
  - SDKs
  - APIs
- 

## Modern Best Practice

Human users should preferably authenticate through **AWS IAM Identity Center** instead of using long-lived IAM users wherever possible.

---

# 5. IAM Groups

## Definition

IAM Groups are collections of IAM Users.

Purpose:

Assign permissions once to the group instead of individually to every user.

---

## Benefits

Instead of:

Developer A → Policy

Developer B → Policy

Developer C → Policy

Create:

Developers Group

↓

Attach Policy Once

↓

All Developers Receive It

---

## Rules

Groups contain only IAM Users.

Groups cannot contain:

- Other Groups
- IAM Roles

Groups cannot authenticate.

---

## Users Can Belong to Multiple Groups

Example:

Alice

↓

Developers

↓

Data Engineers

Permissions from both groups are evaluated together.

---

## 6. IAM Credentials

Credentials prove identity.

Types:

### Console Password

Used for AWS Management Console login.

---

## Access Keys

Used for programmatic access.

Consist of:

- Access Key ID
  - Secret Access Key
- 

## Temporary Credentials

Issued by AWS Security Token Service (STS).

Used with IAM Roles.

Automatically expire.

---

## Multi-Factor Authentication (MFA)

Adds an additional authentication factor.

Recommended methods:

- Authenticator App
  - Hardware Security Key
- 

# 7. Access Keys

Access Keys consist of:

Access Key ID

+

Secret Access Key

Used by:

- AWS CLI
- SDKs
- APIs

---

## Never Hardcode Access Keys

Avoid:

```
ACCESS_KEY="AKIA..."  
SECRET_KEY="..."
```

Instead:

Use IAM Roles whenever possible.

---

## Risks of Exposed Access Keys

If leaked:

- Launch EC2 instances
- Delete S3 buckets
- Read databases
- Create IAM users
- Disable logging
- Exfiltrate data
- Increase AWS costs

Treat leaked credentials as compromised immediately.

---

## 8. Multi-Factor Authentication (MFA)

MFA requires two factors:

1. Password
2. Verification code or hardware key

Benefits:

- Protects against stolen passwords
  - Reduces account takeover risk
  - Recommended for all human users
-

## 9. Password Policies

Organizations should enforce strong password policies.

Typical requirements:

- Minimum length
  - Uppercase letters
  - Lowercase letters
  - Numbers
  - Special characters
  - Password rotation (based on organizational policy)
- 

## 10. Credential Rotation

Long-lived credentials should be rotated regularly.

Recommended process:

1. Create new credentials
2. Update applications
3. Test access
4. Disable old credentials
5. Delete old credentials

Never keep unused access keys.

---

## 11. IAM Roles

### Definition

An IAM Role is an AWS identity that can be temporarily assumed by trusted principals.

Unlike IAM Users, Roles do not belong permanently to a person.

---

### Roles Do Not Have

- Password
- Permanent Access Keys

Instead, AWS issues temporary credentials when the role is assumed.

---

## Services Commonly Using Roles

- EC2
  - Lambda
  - ECS
  - Glue
  - Step Functions
  - Cross-account access
- 

## Example

EC2



IAM Role



Temporary Credentials



S3

No secrets are stored on the EC2 instance.

---

## 12. IAM User vs IAM Role

| IAM User                                    | IAM Role                                              |
|---------------------------------------------|-------------------------------------------------------|
| Permanent identity                          | Temporary identity                                    |
| Has password                                | No password                                           |
| Can have long-lived access keys             | Receives temporary credentials                        |
| Represents a person (or legacy application) | Represents workloads, services, or assumed identities |
| Logs in directly                            | Must be assumed                                       |

---

## 13. Temporary Credentials

Temporary credentials are generated by AWS STS when a role is assumed.

Benefits:

- Automatically expire
- Automatically refreshed by AWS for supported services
- Lower impact if compromised
- No long-lived secrets stored in applications

---

### Why Temporary Credentials Are Better

Long-lived Access Keys:

- Valid until manually rotated
- Higher risk if leaked

Temporary Credentials:

- Time-limited
- Reduced attack window
- Easier to manage securely

---

## 14. Trust Policies

### Definition

A Trust Policy defines **who is allowed to assume an IAM Role**.

It does **not** define permissions on AWS resources.

---

### Trust Policy Answers

Who can use this role?

---

## Example

EC2 Role Trust Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Meaning:

Only the EC2 service is trusted to assume this role.

---

## 15. Permission Policies

Permission Policies define:

What actions are allowed after the role has been assumed.

Examples:

- s3:GetObject
  - s3:PutObject
  - logs:PutLogEvents
  - dynamodb:PutItem
- 

### Trust Policy vs Permission Policy

| Trust Policy             | Permission Policy                    |
|--------------------------|--------------------------------------|
| Who can assume the role? | What actions are allowed?            |
| Uses Principal           | Uses Action and Resource permissions |

Trust Policy

Permission Policy

Attached only to IAM Roles

Can be attached to users, groups, or roles

## 16. Principal

A Principal is the identity requesting access.

Examples:

- IAM User
- IAM Role
- AWS Service (EC2, Lambda)
- AWS Account
- Federated Identity

Example:

```
"Principal": {  
  "Service": "ec2.amazonaws.com"  
}
```

EC2 is the Principal requesting to assume the role.

## 17. Real-World Architecture

Example Data Pipeline:

```
EC2  
|  
▼  
EC2ExecutionRole  
|  
▼  
S3  
  
_____  
  
Lambda  
|  
▼
```

LambdaExecutionRole

|



DynamoDB

Each workload has its own IAM Role.

Benefits:

- Least privilege
- Better auditing
- Smaller blast radius if compromised
- Easier permission management

---

## 18. Security Best Practices

- Never use the Root account for daily work.
- Enable MFA for all human users.
- Prefer IAM Identity Center for workforce access.
- Use IAM Roles instead of IAM Users for AWS workloads.
- Avoid storing Access Keys in code or Git repositories.
- Rotate long-lived credentials regularly.
- Grant only the minimum permissions required (Least Privilege).
- Separate roles for separate workloads.
- Review IAM permissions periodically.
- Monitor activity using AWS CloudTrail.

---

## 19. Common Interview Questions

### Q1. What is IAM?

IAM is the AWS service used to manage authentication and authorization for AWS resources.

---

### Q2. What is the difference between Authentication and Authorization?

Authentication verifies identity.

Authorization determines allowed actions.

---

### Q3. What is the difference between an IAM User and an IAM Role?

An IAM User is a permanent identity that can have passwords and long-lived access keys.

An IAM Role is a temporary identity that is assumed by trusted principals and receives temporary credentials from AWS STS.

---

### Q4. What is a Trust Policy?

A Trust Policy defines which principals are allowed to assume an IAM Role.

---

### Q5. What is a Permission Policy?

A Permission Policy defines what actions are allowed after the identity or role has been granted access.

---

### Q6. What is a Principal?

A Principal is the identity requesting access, such as an IAM User, IAM Role, AWS service, AWS account, or federated identity.

---

### Q7. Why are IAM Roles preferred over IAM Users for EC2?

Because IAM Roles provide temporary credentials, eliminate long-lived access keys on the instance, reduce credential leakage risk, and are automatically managed by AWS.

---

## 20. Key Takeaways

- IAM controls authentication and authorization in AWS.
- IAM Users represent permanent identities.
- IAM Groups simplify permission management.
- IAM Roles provide temporary identities for workloads and services.
- Access Keys should never be hardcoded.
- Prefer temporary credentials over long-lived credentials.
- Trust Policies define **who can assume a role**.
- Permission Policies define **what actions are allowed**.
- The **Principal** is the identity requesting to assume a role.
- Follow the Principle of Least Privilege.
- Modern AWS environments rely heavily on IAM Roles, IAM Identity Center, and temporary credentials instead of long-lived access keys.